

E.3 More Ablation Studies

E.3.1 Memory Budget

We present an ablation study on the parameter budget for the WikiText-103 language modeling task. We show that Scatterbrain outperforms its sparse and low-rank baselines across a range of parameter budgets. The results are presented in Table 5.

Analysis: We have observed that Scatterbrain outperforms its sparse and low-rank baselines under different memory budgets. Similar to what we found in Section 5.2, Performer does not train stably even with $\frac{1}{4}$ of the full attention memory. However, under the Scatterbrain framework, Performer can be combined with Reformer in an elegant way to achieve the same accuracy while using only half of the memory and faster than Reformer by exploiting the sparse+low-rank structure in attention matrices.

Table 5: We run WikiText-103 LM with a sweep of 1/4, 1/8, 1/16 memory budget. We show the validation perplexity and speed-up with respect to the full attention with different efficient Attention layers.

	$\frac{1}{4}$ Mem	$\frac{1}{8}$ Mem	$\frac{1}{16}$ Mem
	Perplexity (Speed-up)	Perplexity	Perplexity
SMYRF	26.76 (1.6 $\times$)	27.68 (1.39 $\times$)	28.7(1.85 $\times$)
PERFORMER	58(2.13 $\times$)	66 (2.01 $\times$)	85(1.77 $\times$)
Scatterbrain	26.26(1.58$\times$)	26.72 (1.87$\times$)	27.74(2.03$\times$)

E.3.2 Different Sparse and Low-rank baselines

Scatterbrain is general enough to accommodate different kinds of sparse and low-rank approximations as its sub-components. In particular, we can combine Local attention or block sparse (from Sparse Transformer and BigBird) + Performer (instead of Reformer + Performer) in a similar fashion. The support of the sparse matrix S will thus be fixed and not adaptive to input, but all the other steps are exactly the same.

We have run additional experiments on the Local attention + Performer combination and BigBird. Recall that in Appendix E, we have shown Scatterbrain can reduce the attention memory of Vision Transformer by 98% at the cost of only 0.8% drop of accuracy when serving as a drop-in replacement for full attention without training on ImageNet. We show the results for local+performer variation with the same memory budget in Table 6.

We have also run additional experiments on Local attention on Copy and Wikitext-103 language modeling task (Table 7). We see that Local attention is reasonably competitive on Wikitext-103 but does not perform well on Copy. The results are not surprising as noted in the Reformer paper that Copy requires non-local attention lookups.

E.3.3 Different Sparse and Low-rank baselines

E.4 Analysis

Recall in Section 5, we have reported the analysis after visualizing the error of REFORMER (sparse), PERFORMER (low-rank), and Scatterbrain (sparse + low-rank) given the same number of parameters when approximating the full attention matrices for each attention layer during training. In Figure 8, we present the visualization.

Table 6: Top-1 Accuracy of pre-trained T2T Vision Transformer on ImageNet with different attention replacements. Error represents the average normalized approximation error to full attention.

Attention	Top-1 Acc
Full Attention	81.7%
SMYRF	79.8%
Local	79.6%
Performer	80.1%
BigBird	80.3%
Scatterbrain (Local + Performer)	80.3%
Scatterbrain (SMYRF + Performer)	80.7%

Table 7: Additional experiments for Local attention on the Copy and Wikitext-103 language modeling task.

Attention	Copy (ppl)	WikiText-103 (ppl)
Full Attention	1	25.258
Reformer	6.8	27.68
Performer	49	66
Local	53	30.72
Scatterbrain	2.58	26.72

The conclusion for language modeling tasks is that sparse+low-rank has the smallest approximation error in most of the cases, and sparse has the largest error, which matches with the end-to-end results. It also confirms the observation in the popular benchmark paper [57] that kernel or low-rank based approximations are less effective for hierarchical structured data. For classification tasks, we again find that Scatterbrain has the smallest approximation error, while PERFORMER is the worst on ListOps and REFORMER has the largest error on classification tasks, which matches with the end-to-end results and confirms our observations earlier (sparse and low-rank approximation excel in different regimes).

E.5 Additional Experiments of Fine-tuning Bert on GLUE

We provide additional experiments of fine-tuning Bert on GLUE in Table 8. We follow the similar setting as [22]. We replace all the attention layers in Bert base model with Scatterbrain and other baselines. Then we fine-tune Bert on 9 downstream tasks for 3 epochs with batch size 32 and learning rate 3e-5. The parameter budget is 1/2 of the full attention because sequence length 128 is not very long. We can see Scatterbrain outperforms all the other baselines in most of the downstream tasks.

Table 8: Results of GLUE when replacing dense attention matrices with SMYRF, PERFORMER and Scatterbrain in BERT base model. We fix the same number of parameters (1/2 of the full) used for approximating the attention matrix for each method.

	CoLA	SST-2	MRPC	STS-B	QQP	MNLI	QNLI	RTE	WNLI
	mcc	acc	acc	corr	acc	acc	acc	acc	acc
FULL	0.576	0.934	0.874	0.879	0.905	0.813	0.916	0.668	0.43
SMYRF	0.538	0.912	0.833	0.856	0.898	0.775	0.879	0.626	0.412
PERFORMER	0.508	0.838	0.782	0.203	0.831	0.563	0.763	0.556	0.449
Scatterbrain	0.569	0.927	0.863	0.867	0.902	0.813	0.893	0.619	0.428

F Further Discussions and Future Work

In this paper, we present Scatterbrain, unifying the strength of sparse and low-rank approximation. It is inspired by the observations on the attention matrix structures induced by the data and softmax function as well as the classical robust-PCA algorithm. In our implementation and analysis, we have REFORMER/SmYrf and PERFORMER as the back-bone for sparse and low-rank approximations because of their properties, e.g. Performer is unbiased. Scatterbrain is fundamentally a framework for combining the strength of sparse and low-rank variants, so it can be easily extended to other variants, such as Routing Transformer [53] or Nystromformer [67]. Further more, our observations on the connection between entropy and low-rank/sparse approximation error also provide an opportunity for efficiently detecting the approximation or compression method to choose for different architectures or benchmarks.